

Up next in Hands-on Labs →

Jenkins Labs

 Hands-on lab · 60 min · Env: See book overview · Overview & prereqs · Download PDF ·

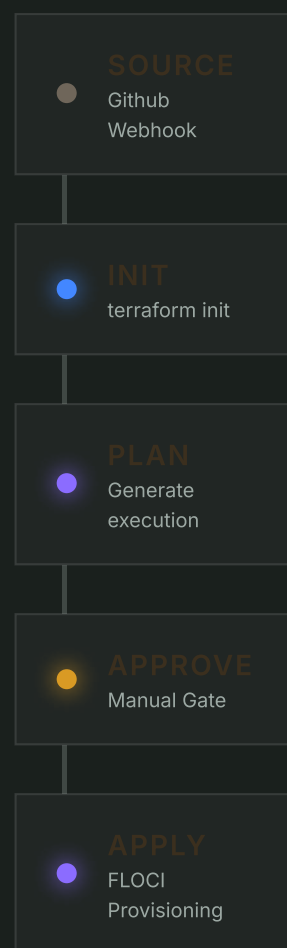
Reset checks

CI/CD ENGINEERING

Infrastructure Automation:
Terraform & Simulated Cloud

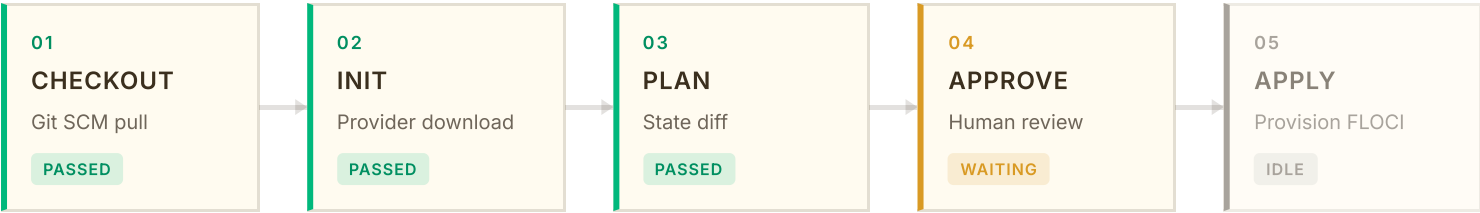
JENKINS PIPELINES
TERRAFORM WORKFLOWS
SIMULATED AWS (FLOCI)

Nabawy



01 / The Infrastructure Pipeline

Infrastructure as Code (IaC) requires the same rigorous CI/CD practices as application code. However, unlike application deployments, infrastructure changes can cause immediate, irreversible state shifts. The pipeline must therefore include strict validation and manual approval gates before applying changes.



WHAT

A Jenkins pipeline that executes Terraform commands against a simulated AWS environment (FLOCI) to provision VPCs, subnets, and compute resources.

WHEN

Triggered automatically by a GitHub webhook on push, but pauses for manual approval before the destructive `apply` stage.

WHY

To practice infrastructure automation safely. FLOCI mimics the AWS API without incurring real costs, allowing teams to test CI/CD workflows aggressively.

WHAT CHANGES

Infrastructure moves from manual CLI execution to a versioned, peer-reviewed, automated pipeline. State shifts become deliberate, logged events.

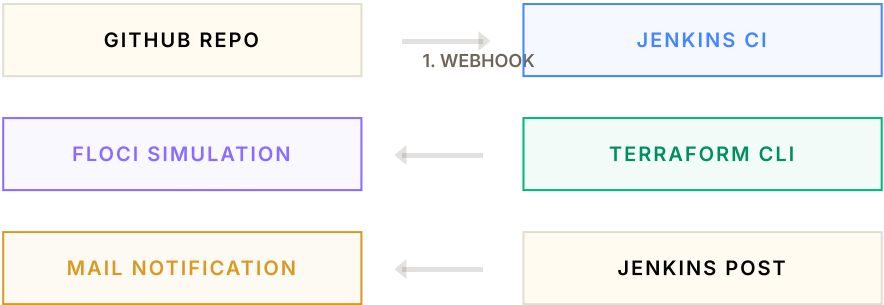
WHY THIS EXISTS

Without a pipeline, infrastructure drifts. Engineers apply changes locally, state files diverge, and recovery becomes impossible. A pipeline enforces a single, auditable path from code to cloud.

02 / Architecture & Pre-Flight

The pipeline bridges GitHub, Jenkins, and FLOCI. The critical integration point is the `Provider.tf` file, which redirects standard AWS API calls to the FLOCI simulation endpoint.

EXECUTION ARCHITECTURE



PRE-FLIGHT VERIFICATION

```
$ terraform -version
Terraform v1.5.0
$ curl -s -o /dev/null -w "%{http_code}" http://localhost:8080
200
$ curl -s -o /dev/null -w "%{http_code}" http://floci-sim.local
200
```

GOTCHA

FLOCI is a simulation, not real AWS. Standard AWS credential patterns (IAM roles, `~/.aws/credentials`) do not apply. The `Provider.tf` file must be explicitly configured with the FLOCI endpoint URL and simulation tokens gathered in Step 05 | 03 / 08

03 / Workspace Isolation

Infrastructure state must be isolated per environment. A change in `dev` must never alter `prod` state. Terraform workspaces combined with environment-specific `.tfvars` files achieve this within a single pipeline.

ENVIRONMENT TRACKS

DEV	Workspace: dev File: dev.tfvars	ACTIVE
STG	Workspace: stg File: stg.tfvars	IDLE
PROD	Workspace: prod File: Prod.tfvars	LOCKED

WORKSPACE SELECTION LOGIC

```
sh 'terraform workspace select ${ENV_NAME} || terraform workspace new ${ENV_NAME}'
Switched to workspace "dev".
```

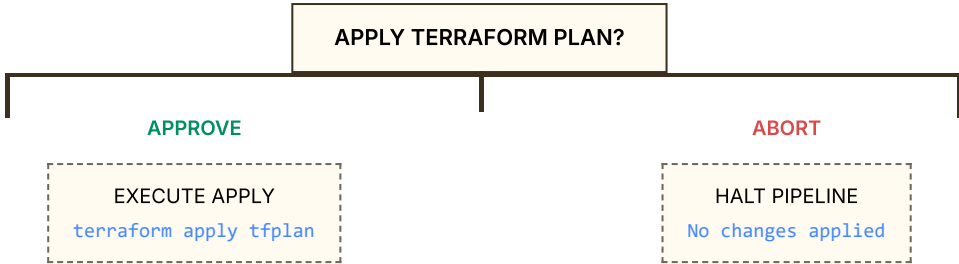
INTERVIEW CHECK

Q: Why use workspaces instead of separate directories for state?

A: Workspaces allow the same Terraform configuration to manage multiple distinct state files without duplicating code. It simplifies the pipeline by parameterizing the environment via a simple `--workspace` flag rather than altering backend paths dynamically.

04 / The Approval Gate

Terraform `apply` is destructive. A pipeline that auto-applies infrastructure changes is dangerous. The pipeline must pause after the `plan` stage, presenting the execution plan for human review before proceeding.



JENKINS INPUT STEP

```
stage('Approve') {
  steps {
    input message: 'Apply this Terraform plan?', ok: 'Approve'
  }
}
```

GOTCHA

If the pipeline is aborted at the `input` step, Jenkins marks the build as `ABORTED`, not `FAILED`. Ensure your `post` actions distinguish between failure (requiring a mail alert) and abort (a deliberate human choice).

WHY IT MATTERS

Automation stops here because a human decision is required. The `plan` output is the contract; the `apply` is the execution. Skipping this gate removes accountability from the infrastructure lifecycle.

05 / FLOCI Provider Configuration

To target FLOCI instead of real AWS, the `Provider.tf` must override the default AWS endpoints. This intercepts the Terraform AWS provider's HTTP calls and routes them to the local simulation.

```
# Provider.tf
provider "aws" {
  region = "us-east-1"
  # Redirect to FLOCI Simulation
  endpoints {
    ec2 = "http://floci-sim.local/ec2"
    s3 = "http://floci-sim.local/s3"
    elasticloadbalancing = "http://floci-sim.local/elb"
  }
  skip_credentials_validation = true
  skip_metadata_api_check = true
  skip_requesting_account_id = true
}
```

COMMAND

```
$ terraform init
```

RESULT

```
Initializing the backend...
Initializing provider plugins...
Terraform has been successfully initialized!
```

WHY IT MATTERS

By skipping credential validation and overriding endpoints, Terraform operates in a sandbox. This allows engineers to test complex VPC topologies and EC2 deployments against a mock API, ensuring the IaC logic is sound before touching a real cloud billing account.

06 / Lab: End-to-End Provisioning

Execute the pipeline and observe the state transitions, from webhook trigger through manual approval to simulated deployment and failure recovery.

- OBSERVE

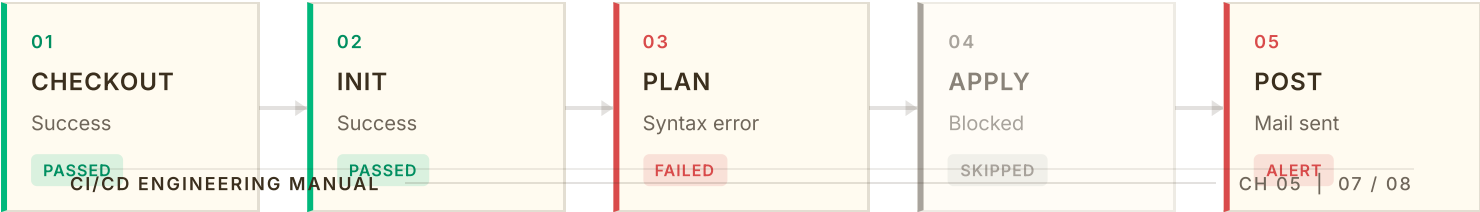
The GitHub repo contains the Terraform code and Jenkinsfile. The webhook is active. We need to verify the pipeline reaches the approval gate.
- EXECUTE

Push a minor change to trigger the pipeline. Watch the stages: Checkout → Init → Workspace Select → Plan. The pipeline will pause at "Approve".
- VERIFY

Review the `plan` output in the Jenkins logs. Click "Approve". Confirm the `apply` stage runs and FLOCI reflects the new VPC resources.
- PROOF

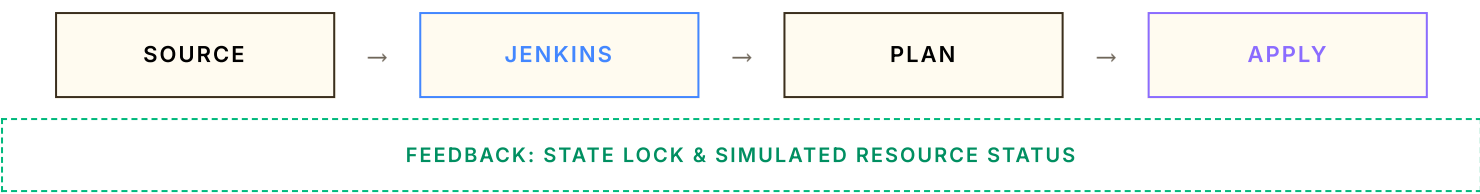
Introduce a syntax error in `VPC.tf` and push. The pipeline will fail at `Plan`. The `post { failure { mail ... } }` block executes, sending an alert.

FAILURE & RECOVERY FLOW



07 / System Map & Troubleshooting

Infrastructure pipelines fail in predictable ways: state conflicts, provider mismatches, or unauthorized credentials. Use this map to isolate the failure domain.



TROUBLESHOOTING FLOW

Init fails (provider)	FLOCI endpoint unreachable	Verify FLOCI service is running and URL in Provider.tf	
SYMPTOM	ROOT CAUSE	RECOVERY	
Apply fails (403)	FLOCI simulation token invalid	Update Jenkins credentials for FLOCI provider	
Pipeline paused forever	Approval gate ignored	Review Jenkins input prompt or abort build	

INTERVIEW CHECK

Q: Why use FLOCI instead of a real AWS Sandbox account?

A: Real AWS sandboxes still incur costs and carry the risk of accidental production impact if IAM boundaries are misconfigured. FLOCI provides a zero-cost, zero-risk API mock that is ideal for testing the CI/CD pipeline mechanics themselves.

Up next in Hands-on Labs →

Jenkins Labs

RELATED BOOKS

Jenkins Labs
180 min · **ADV**
→ Next in path

Lab 04: Reusable Pipeline Logic
55 min · **ADV**
→ Previous in path

IaC for Environments
30 min · **INT**
→ Prerequisite

Lab 01: Automated Integration Foundations
45 min · **INT**
→ Shared tags